BrightLearn
AI & DATA SKILLS

STUDENT EXERCISE · SQL JOINS

SQL Joins Exercise

Practice INNER JOIN, LEFT JOIN, and FULL OUTER JOIN on a five-table retail dataset. Fifteen questions, three joins, one bonus challenge — write the SQL.

TOPIC	QUESTIONS
SQL Joins	15 + 5 bonus
LEVEL	**INCLUDES**
Beginner	5 tables · Expected output

Inner · Left · Full Outer

00 Welcome to the Exercise

This exercise gives you fifteen real SQL questions to practise the three most important joins — INNER, LEFT, and FULL OUTER. You'll work with five tables from a streaming-service business, write your own SQL, and check your answers against the expected output.

DEFINITION

How this exercise works

Five tables. Fifteen questions across three parts. **Part A is INNER JOIN, Part B is LEFT JOIN, Part C is FULL OUTER JOIN**. Each question tells you which tables to use, the join to apply, and the expected output columns. Write your SQL in the box under each question, then run it on your own database to check. A bonus challenge at the end lets you reason about the data without writing more SQL.

★ WHAT YOU WILL PRACTISE

- **Matching rows across tables** using a common key (an INNER JOIN).
- **Keeping every row from one side** — even rows with no match (a LEFT JOIN).
- **Keeping every row from both sides** — including unmatched rows on either end (a FULL OUTER JOIN).
- **Reading expected output** to check your work — a skill every analyst needs.
- **Reasoning about missing data** — which is what the bonus questions test.

★ HOW TO SUBMIT YOUR ANSWERS

Set up the five tables in your favourite SQL environment — SQLite, PostgreSQL, MySQL, SQL Server, BigQuery, or Databricks all work. Write each query in a single .sql file numbered Q1 to Q15. Add comments above each one. Run them and screenshot the results. Submit the .sql file plus screenshots through the BrightLearn portal.

01 The Three Joins

Before you write a single query, make sure these three patterns are clear in your head. The same two tables joined in three different ways give you three different answers — and choosing the right one is half the skill.

01 · INNER JOIN

Returns only rows where the join condition is true on BOTH sides. Unmatched rows are dropped.

```
SELECT  ...
FROM    customers c
INNER JOIN  orders o  ON c.customer_id = o.customer_id;

→ only customers who placed orders, and only orders with a matching customer.
```

02 · LEFT JOIN

Returns every row from the LEFT table. Matching rows from the right are included; unmatched rows on the right become NULL.

```
SELECT  ...
FROM    customers c
LEFT JOIN   orders o  ON c.customer_id = o.customer_id;

→ every customer, with or without orders. Customers without orders get NULL on
the order columns.
```

03 · FULL OUTER JOIN

Returns every row from BOTH tables. Unmatched rows on either side become NULL on the other side's columns.

```
SELECT  ...
FROM    customers c
FULL OUTER JOIN  orders o  ON c.customer_id = o.customer_id;

→ every customer AND every order. Useful to find unmatched rows on either side.
```

★ THE MENTAL PICTURE

Picture two overlapping circles. INNER JOIN is the overlap. LEFT JOIN is the left circle plus the overlap. FULL OUTER JOIN is both circles, overlap and all. Once you see this picture clearly, picking the right join becomes automatic.

02 The Five Tables

These are the five tables you'll be querying. Read them carefully — some rows are intentionally missing matches in other tables, which is exactly what makes the three joins produce different results.

Table 1 · users

user_id	user_name	country
1	Nomvula	Johannesburg
2	David	Cape Town
3	Anele	Durban
4	Kabelo	Pretoria
5	Lerato	Port Elizabeth

Table 2 · plans

plan_id	plan_name	monthly_price
10	Basic	79
11	Standard	129
12	Premium	199
13	Family	249
14	Mobile	59

Table 3 · subscriptions

subscription_id	user_id	plan_id	start_date
501	1	10	2026-01-15
502	2	11	2026-02-01
503	1	12	2026-03-10
504	6	11	2026-03-20
505	3	13	2026-04-05

Table 4 · shows

show_id	show_title	genre
701	Comedy Hour	Comedy

702	Crime Time	Drama
703	Tech Tales	Documentary
704	Cooking Lab	Lifestyle
706	Wild Earth	Documentary

Table 5 · viewing_sessions

session_id	user_id	show_id	watch_minutes
901	1	701	45
902	2	703	30
903	1	702	60
904	7	701	20
905	3	705	90

⚠ **LOOK CAREFULLY— THESE GAPS ARE ON PURPOSE**

Some rows reference IDs that do not exist in the parent table: **subscription 504 → user 6 (doesn't exist)**, **session 904 → user 7 (doesn't exist)**, and **session 905 → show 705 (doesn't exist)**. Some parent rows have no children: users 4 and 5 have no subscriptions or sessions; plan 14 (Mobile) has no subscribers; shows 704 and 706 were never watched. These gaps make INNER, LEFT, and FULL OUTER joins return visibly different results — which is the whole point of the exercise.

03 Part A — INNER JOIN

The first five questions all use INNER JOIN — return only the rows where both sides match. Notice how rows with missing matches disappear from your result set.

QUESTION 01

Show every user who has a subscription. Match users to subscriptions.

TABLES TO USE

`users` · `subscriptions`

EXPECTED COLUMNS

`user_id, user_name, subscription_id, start_date`

WRITE YOUR SQL HERE

```sql
select u.user_id,
       u.user_name,
       s.subscription_id,
       s.start_date
 from `workspace`.`joins`.`users` u
 inner join `workspace`.`joins`.`subscriptions` s
 on u.user_id = s.user_id;
```

QUESTION 02

Show every subscription with its matching plan name and monthly price.

TABLES TO USE

`subscriptions` · `plans`

EXPECTED COLUMNS

`subscription_id, user_id, plan_name, monthly_price`

WRITE YOUR SQL HERE

```sql
select subscription_id,
       user_id,
       plan_name,
       monthly_price
 from   `workspace`.`joins`.`subscriptions`
 join `workspace`.`joins`.`plans`
 on subscriptions.plan_id = plans.plan_id;
```

QUESTION 03

Show every viewing session that has a matching show. Include the show title and genre.

TABLES TO USE

`viewing_sessions` · `shows`

EXPECTED COLUMNS

`session_id, user_id, show_title, genre, watch_minutes`

WRITE YOUR SQL HERE

```sql
select session_id,
       user_id,
       show_title,
       genre,
       watch_minutes
 from `workspace`.`joins`.`viewing_sessions`
 inner join `workspace`.`joins`.`shows`
 on shows.show_id = viewing_sessions.show_id;
```

QUESTION 04

Show every viewing session with the user who watched it. Only show sessions with a matching user.

TABLES TO USE

`users` · `viewing_sessions`

EXPECTED COLUMNS

`user_name, country, session_id, show_id, watch_minutes`

WRITE YOUR SQL HERE

```sql
select user_name,
       country,
       session_id,
       show_id,
       watch_minutes
 from `workspace`.`joins`.`users`
 join `workspace`.`joins`.`viewing_sessions`
 on users.user_id = viewing_sessions.user_id;
```

QUESTION 05

Show users along with their subscriptions, the plan name, and the price. Use only users who have both a subscription and a valid plan.

TABLES TO USE

`users` · `subscriptions` · `plans`

EXPECTED COLUMNS

`user_name, country, plan_name, monthly_price, start_date`

WRITE YOUR SQL HERE

```sql
select user_name,
       country,
       plan_name,
       monthly_price,
       start_date
 from `workspace`.`joins`.`users`
 join `workspace`.`joins`.`subscriptions`
 on users.user_id = subscriptions.user_id
 join `workspace`.`joins`.`plans`
 on subscriptions.plan_id = plans.plan_id;
```

04 Part B — LEFT JOIN

These five questions use LEFT JOIN — every row from the left-hand table is kept, even when it has no match on the right. Watch where NULL values appear in your result.

QUESTION 06

Show every user and any subscriptions they have. Users without subscriptions must still appear.

TABLES TO USE

`users` · `subscriptions`

EXPECTED COLUMNS

`user_id, user_name, subscription_id, start_date`

WRITE YOUR SQL HERE

```sql
select u.user_id,
       u.user_name,
       s.subscription_id,
       s.start_date
 from `workspace`.`joins`.`users`u
 left join `workspace`.`joins`.`subscriptions`s
 on u.user_id = s.user_id;
```

QUESTION 07

Show every plan and the subscriptions on it. Plans with no subscribers must still appear.

TABLES TO USE

`plans` · `subscriptions`

EXPECTED COLUMNS

`plan_id, plan_name, subscription_id, user_id`

WRITE YOUR SQL HERE

```sql
select p.plan_id,
       p.plan_name,
       s.subscription_id,
       s.user_id
 from `workspace`.`joins`.`plans`p
 left join `workspace`.`joins`.`subscriptions`s
 on p.plan_id = s.plan_id;
```

QUESTION 08

Show every show and any viewing sessions on it. Shows that were never watched must still appear.

TABLES TO USE

`shows` · `viewing_sessions`

EXPECTED COLUMNS

`show_id, show_title, session_id, watch_minutes`

WRITE YOUR SQL HERE

```sql
select s.show_id,
       s.show_title,
       v.session_id,
       v.watch_minutes
 from `workspace`.`joins`.`viewing_sessions`v
 left join `workspace`.`joins`.`shows`s
 on v.show_id = s.show_id;
```

QUESTION 09

Show every viewing session and the user who watched it. Sessions referencing users that do not exist must still appear (with NULL user details).

TABLES TO USE

`viewing_sessions` · `users`

EXPECTED COLUMNS

`session_id, show_id, watch_minutes, user_id, user_name`

WRITE YOUR SQL HERE

```
select v.session_id,
       v.show_id,
       v.watch_minutes,
       u.user_id,
       u.user_name
from `workspace`.`joins`.`viewing_sessions`v
left join `workspace`.`joins`.`users`u
on v.user_id = u.user_id;
```

QUESTION 10

Show every user, the plan they are on (if any), and the monthly price. Users without a subscription must still appear.

TABLES TO USE

users · subscriptions · plans

EXPECTED COLUMNS

user_name, country, plan_name, monthly_price

WRITE YOUR SQL HERE

```
select u.user_name,
       u.country,
       p.plan_name,
       p.monthly_price
from `workspace`.`joins`.`users`u
left join `workspace`.`joins`.`subscriptions`s
on u.user_id = s.user_id
left join `workspace`.`joins`.`plans`p
on s.plan_id = p.plan_id;
```

05 Part C — FULL OUTER JOIN

The final five questions use FULL OUTER JOIN — every row from both tables, matched where possible and NULL elsewhere. This is the join that reveals everything missing on both sides at once.

> ★ **SQL DIALECT NOTE**
>
> PostgreSQL, SQL Server, Oracle, and most warehouses support FULL OUTER JOIN directly. MySQL does not — emulate it by combining a LEFT JOIN and a RIGHT JOIN with UNION. If you are practising on MySQL, write the LEFT + RIGHT + UNION pattern for these questions.

QUESTION 11

Show every user and every subscription, including users without subscriptions AND subscriptions referencing users that do not exist.

TABLES TO USE

`users` · `subscriptions`

EXPECTED COLUMNS

`user_id, user_name, subscription_id, start_date`

WRITE YOUR SQL HERE

```sql
select u.user_id,
       u.user_name,
       s.subscription_id,
       s.start_date
from `workspace`.`joins`.`users`u
full outer join `workspace`.`joins`.`subscriptions`s
on u.user_id = s.user_id;
```

QUESTION 12

Show every plan and every subscription, including plans without subscribers AND any

subscription referencing a plan that does not exist.

TABLES TO USE

`plans` · `subscriptions`

EXPECTED COLUMNS

`plan_id, plan_name, subscription_id, user_id`

WRITE YOUR SQL HERE

```sql
select p.plan_id,
       p.plan_name,
       s.subscription_id,
       s.user_id
from `workspace`.`joins`.`plans`p
full outer join `workspace`.`joins`.`subscriptions`s
on p.plan_id = s.plan_id;
```

QUESTION 13

Show every show and every viewing session, including shows that were never watched AND sessions referencing shows that do not exist.

TABLES TO USE

`shows` · `viewing_sessions`

EXPECTED COLUMNS

`show_id, show_title, session_id, watch_minutes`

WRITE YOUR SQL HERE

```sql
select s.show_id,
       s.show_title,
       v.session_id,
       v.watch_minutes
from `workspace`.`joins`.`viewing_sessions`v
full outer join `workspace`.`joins`.`shows`s
on v.show_id = s.show_id;
```

QUESTION 14

Show every user and every viewing session, including users with no sessions AND sessions referencing users who do not exist.

TABLES TO USE

`users` · `viewing_sessions`

EXPECTED COLUMNS

`user_id, user_name, session_id, show_id, watch_minutes`

WRITE YOUR SQL HERE

```sql
select u.user_id,
       u.user_name,
       v.session_id,
       v.show_id,
       v.watch_minutes
from `workspace`.`joins`.`users`u
full outer join `workspace`.`joins`.`viewing_sessions`v
on u.user_id = v.user_id;
```

QUESTION 15

Show every user, every subscription, and every plan in one query — using FULL OUTER JOIN throughout. This is the hardest question — get all gaps visible at once.

TABLES TO USE

`users` · `subscriptions` · `plans`

EXPECTED COLUMNS

`user_id, user_name, subscription_id, plan_id, plan_name`

WRITE YOUR SQL HERE

```sql
select u.user_id,
       u.user_name,
       s.subscription_id,
       p.plan_id,
       p.plan_name
from `workspace`.`joins`.`users`u
      left join `workspace`.`joins`.`subscriptions`s
on u.user_id = s.user_id
left join `workspace`.`joins`.`plans`p
on s.plan_id = p.plan_id;
```

06 Bonus Challenge

Now the conceptual test. Without writing any more SQL, answer these five questions from looking at the tables — they prove you really understand how the joins work and what they reveal about your data.

BONUS 01

Which users have not subscribed to any plan?

WRITE YOUR SQL HERE

```
KABELO&LERATO
```

BONUS 02

Which subscriptions reference users that do not exist in the users table?

WRITE YOUR SQL HERE

```
NULL
```

BONUS 03

Which shows have never been watched?

WRITE YOUR SQL HERE

```
NULL
```

BONUS 04

Which viewing sessions reference shows that do not exist?

WRITE YOUR SQL HERE

```
NULL
```

BONUS 05

Which plans have no subscribers?

WRITE YOUR SQL HERE

```
MOBILE
```

07 Self-Check & Submission

Before you submit, walk through this checklist. The students who score highest are the ones who check their own work first — because half the learning happens in the verification.

★ BEFORE YOU SUBMIT — THE CHECKLIST

- **Every query runs without errors** on your SQL environment.
- **Output columns match the spec** — same names, same order.
- **INNER JOIN questions return fewer rows** than LEFT or FULL — confirm by counting.
- **LEFT JOIN results contain NULLs** where the right-hand match was missing.
- **FULL OUTER JOIN results show NULLs on both sides** for the unmatched rows.
- **Your bonus answers reference specific IDs** from the tables — e.g. "users 4 and 5", not just "some users".

WHAT TO SUBMIT

★ YOUR SUBMISSION PACK

- **One .sql file** containing all 15 queries, each clearly labelled (-- Q1, -- Q2, etc.).
- **Screenshots** of the result of each query (or paste-in text output).
- **Bonus answers** in a separate text file or at the bottom of the .sql file.
- **A short note** describing which SQL environment you used (PostgreSQL, SQLite, etc.).

★ ONE LAST THING

The fastest way to consolidate what you've learned is to explain it to someone else. After you submit, find a classmate or a friend and walk them through one INNER JOIN, one LEFT JOIN, and one FULL OUTER JOIN from this exercise. If you can teach it, you own it.

Five tables.
Three joins.

Master INNER JOIN, LEFT JOIN, and FULL OUTER JOIN by writing real SQL on real tables. Fifteen questions plus a bonus — then check your answers against the expected output.

★ BRIGHTLEARN · SQL JOINS EXERCISE ★